Legacy Forms of Subroutines

Version 2026.1
2026-04-20

InterSystems Corporation One Congress Street Boston MA 02114 www.intersystems.com

Legacy Forms of Subroutines
PDF generated on 2026-04-20
InterSystems IRIS Version 2026.1

For Support questions about any InterSystems products, contact:

InterSystems Worldwide Response Center (WRC)
Tel: +1-617-621-0700
Tel: +44 (0) 844 854 2917
Email: support@InterSystems.com

Table of Contents

Legacy Forms of Subroutines

A routine can contain multiple subroutines, using the term *subroutine* in a generic sense. InterSystems recommends that in any new routines you may create, you define procedures. In existing code, you may see subroutines of other forms. This page describes these other forms and explains how to invoke them, if needed.

1 Recognizing Legacy Forms

Legacy forms of subroutines use labels but are not enclosed within curly braces. The following list shows the possible syntaxes with their formal names, for reference purposes. In all cases, *label* is the identifier for the unit of code, *args* is the argument list, and the optional *scopekeyword* is either `Public` or `Private`.

subroutine

Formally, a true subroutine (as opposed to a subroutine in a generic sense) is a unit of code of the following form:

```
label(args) scopekeyword
      //implementation
  QUIT
```

See Subroutines.

extrinsic function

Formally, a true extrinsic function (as opposed to a system-defined ObjectScript intrinsic function) is a unit of code of the following form:

```
label(args) scopekeyword
      //implementation
  QUIT optionalreturnvalue
```

A function returns a value, unlike a subroutine. See Functions.

In these legacy forms, variables defined within them are available after the subroutine or function finishes execution. Consequently, these legacy forms use different techniques to manage variable scope—specifically the NEW and KILL commands.

2 Subroutines

2.1 Syntax

Subroutine syntax:

```
label [ ( param [ = default  ][ , ...] ) ]
   code
   QUIT
```

Invoking syntax:

```
DO label [ ( param [ , ...]  ) ]
```

or

```
GOTO label
```

Argument	Description
label	The name of the subroutine. A standard label. It must start in column one. The parameter parentheses following the label are optional. If specified, the subroutine cannot be invoked using a **GOTO** call. Parameter parentheses prevent code execution from "falling through" into a subroutine from the execution of the code that immediately precedes it. When InterSystems IRIS encounters a label with parameter parentheses (even if they are empty) it performs an implicit **QUIT**, ending execution rather than continuing to the next line in the routine.
param	The parameter value(s) passed from the calling program to the subroutine. A subroutine invoked using the **GOTO** command cannot have *param* values, and must not have parameter parentheses. A subroutine invoked using the **DO** command may or may not have *param* values. If there are no *param* values, empty parameter parentheses may be specified or omitted. Specify a *param* variable for each parameter expected by the subroutine. The expected parameters are known as the *formal parameter list.*. There may be none, one, or more than one *param*. Multiple *param* values are separated by commas. InterSystems IRIS automatically invokes **NEW** on the referenced *param* variables. Parameters may be passed by value or by reference.
default	An optional default value for the *param* preceding it. You can either provide or omit a default value for each parameter. A default value is applied when no actual parameter is provided for that formal parameter, or when an actual parameter is passed by reference and the local variable in question does not have a value. This default value must be a literal: either a number, or a string enclosed in quotation marks. You can specify a null string ("") as a default value. This differs from specifying no default value, because a null string defines the variable, whereas the variable for a parameter with no specified or default value would remain undefined. If you specify a default value that is not a literal, InterSystems IRIS issues a <PARAMETER> error.
code	A block of code. This block of code is normally accessed by invoking the label. However, it can also be entered (or reentered) by calling another label within the code block or issuing a label + offset **GOTO** command. A block of code can contain nested calls to other subroutines, functions, or procedures. It is recommended that such nested calls be performed using **DO** commands or function calls, rather than a linked series of **GOTO** commands. This block of code is normally exited by an explicit **QUIT** command; this **QUIT** command is not always required, but is a recommended coding practice. You can also exit a subroutine by using a **GOTO** to an external label.

2.2 Description

A subroutine is a block of code identified by a label found in the first column position of the first line of the subroutine. Execution of a subroutine most commonly completes by encountering an explicit **QUIT** statement.

A subroutine is invoked by either the **DO** command or the **GOTO** command.

- A **DO** command executes a subroutine and then resumes execution of the calling routine. Thus, when InterSystems IRIS encounters a **QUIT** command in the subroutine, it returns to the calling routine to execute the next line following the **DO** command.
- A **GOTO** command executes a subroutine but does not return control to the calling program. When InterSystems IRIS encounters a **QUIT** command in the subroutine, execution ceases.

You can pass parameters to a subroutine invoked by the **DO** command; you cannot pass parameters to a subroutine invoked by the **GOTO** command. You can pass parameters by value or by reference. See Passing Arguments.

The same variables are available to a subroutine and its calling routine.

A subroutine does not return a value.

3 Functions

A function, by default and recommendation, is a procedure. You can, however, define a function that is not a procedure. This section describes such functions.

3.1 Syntax

Non-procedure function syntax:

```
label ( [param [ = default  ]] [ , ...] )
   code
   QUIT expression
```

Invoking syntax:

```
command $$label([param[ ,...]])
```

or

```
DO label([param[ ,...]])
```

Argument	Description
label	The name of the function. A standard label. It must start in column one. The parameter parentheses following the label are mandatory.
param	A variable for each parameter expected by the function. The expected parameters are known as the *formal parameter list* . There may be none, one, or more than one *param*. Multiple *param* values are separated by commas. InterSystems IRIS automatically invokes **NEW** for the referenced *param* variables. Parameters may be passed by value or by reference.
default	An optional default value for the *param* preceding it. You can either provide or omit a default value for each parameter. A default value is applied when no actual parameter is provided for that formal parameter, or when an actual parameter is passed by reference and the local variable in question does not have a value. This default value must be a literal: either a number, or a string enclosed in quotation marks. You can specify a null string () as a default value. This differs from specifying no default value, because a null string defines the variable, whereas the variable for a parameter with no specified or default value would remain undefined. If you specify a default value that is not a literal, InterSystems IRIS issues a <PARAMETER> error.
code	A block of code. This block of code can contain nested calls to other functions, subroutines, or procedures. Such nested calls must be performed using **DO** commands or function calls. You cannot exit a function's code block by using a **GOTO** command. This block of code can only be exited by an explicit **QUIT** command with an expression.
expression	The function's return value, specified using any valid ObjectScript expression. The **QUIT** command with expression is a mandatory part of a user-defined function. The value that results from expression is returned to the point of invocation as the result of the function.

3.2 Description

User-defined functions are described in this section. Calls to user-defined functions are identified by a $$ prefix. (A user-defined function is also known as an *extrinsic function*.)

User-defined functions allow you to add functions to those supplied by InterSystems IRIS. Typically, you use a function to implement a generalized operation that can be invoked from any number of programs.

A function is always called from within an ObjectScript command. It is evaluated as an expression and returns a single value to the invoking command. For example:

ObjectScript

```
SET x=$$myfunc()
```

4 Legacy Code and Labels

A procedure is defined by a label and curly braces, which encapsulate the implementation. In contrast, for the legacy forms of subroutines shown on this page, there is no automatic encapsulation of the code. That is, a label provides an entry point, but it does not define an encapsulated unit of code. This means that once the labelled code executes, execution continues

into the next labelled unit of code unless execution is stopped or redirected elsewhere. There are three ways to stop execution of a unit of code:

- Execution encounters a QUIT or RETURN.
- Execution encounters the closing curly brace (“}”) of a TRY. When this occurs, execution continues with the next line of code following the associated **CATCH** block.
- Execution encounters the next procedure block (a label with parameter parentheses). Execution stops when encountering a label line with parentheses, even if there are no parameters within the parentheses.

In the following example, code execution continues from the code under label0 to that under label1:

ObjectScript

```
  SET x = $RANDOM(2)
  IF x=0 {DO label0
          WRITE "Finished Routine0",! }
  ELSE {DO label1
          WRITE "Finished Routine1",! }
  QUIT
label0
  WRITE "In Routine0",!
  FOR i=1:1:5 {
      WRITE "x = ",x,!
      SET x = x+1 }
  WRITE "At the end of Routine0",!
label1
  WRITE "In Routine1",!
  FOR i=1:1:5 {
      WRITE "x = ",x,!
      SET x = x+1 }
  WRITE "At the end of Routine1",!
```

In the following example, the labeled code sections end with either a QUIT or RETURN command. This causes execution to stop. Note that **RETURN** always stops execution, **QUIT** stops execution of the current context:

ObjectScript

```
  SET x = $RANDOM(2)
  IF x=0 {DO label0
          WRITE "Finished Routine0",! }
  ELSE {DO label1
          WRITE "Finished Routine1",! }
  QUIT
label0
  WRITE "In Routine0",!
  FOR i=1:1:5 {
      WRITE "x = ",x,!
      SET x = x+1
      QUIT }
  WRITE "Quit the FOR loop, not the routine",!
  WRITE "At the end of Routine0",!
  QUIT
  WRITE "This should never print"
label1
  WRITE "In Routine1",!
  FOR i=1:1:5 {
      WRITE "x = ",x,!
      SET x = x+1 }
  WRITE "At the end of Routine1",!
  RETURN
  WRITE "This should never print"
```

In the following example, the second and third labels identify procedure blocks (a label specified with parameter parentheses). Execution stops when encountering a procedure block label:

ObjectScript

```
  SET x = $RANDOM(2)
  IF x=0 {DO label0
          WRITE "Finished Routine0",! }
  ELSE {DO label1
          WRITE "Finished Routine1",! }
  QUIT
label0
  WRITE "In Routine0",!
  FOR i=1:1:5 {
      WRITE "x = ",x,!
      SET x = x+1 }
  WRITE "At the end of Routine0",!
label1()
  WRITE "In Routine1",!
  FOR i=1:1:5 {
      WRITE "x = ",x,!
      SET x = x+1 }
  WRITE "At the end of Routine1",!
label2()
  WRITE "This should never print"
```